



**NATIONAL UNIVERSITIES OF MODERN
LANGUAGES
FACULTY OF ENGINEERING AND COMPUTING
DEPARTMENT OF SOFTWARE ENGINEERING**



To be filled by student

Course Name : SOFTWARE QUALITY ENGINEERING

Assignment No./Presentation/Project CCP

Date : 23 May, 2025

Submitted by

Class: BSSE (VI) Section -- Session (M/A) Afternoon

Name: Zulgarnain Hassan Roll No. SP21379

Name: Mumtaz Ali Roll No. SP21386

Name: Muhammad Younus Roll No. SP21452

Submitted to

Instructor Name: Dr. Sumaira Nazir

To be filled by Instructor

Instructor Comments:

Marks Obtained: _____ Total Marks: _____

CLO No. : _____

Software Quality Assurance Plan (SQAP) for BreatheWell

1. Purpose and Scope

The purpose of this Software Quality Assurance Plan (SQAP) is to define the specific activities, responsibilities, and procedures for ensuring the quality of the **BreatheWell** AI-powered web application. This plan addresses the systematic approach to quality assurance throughout the entire software development lifecycle of BreatheWell, from initial requirements gathering and design through development, testing, deployment, and maintenance.

The scope of this SQAP encompasses all components of the BreatheWell system, including:

- The **React.js frontend** for an intuitive user interface.
- The **Python backend** (using Django or Flask) for API development, server-side logic, and machine learning model integration.
- **PostgreSQL and MongoDB databases** for efficient data management.
- **Firebase** for user authentication and real-time communication.
- All essential features as described in the project overview: Symptom Submission, AI-Powered Cough & Breathing Analysis, Smart Diagnosis System, Symptom Tracker & Dashboard, Personalized Care Plan, Environmental Alert System, AI Chatbot, Doctor Appointment, Patient History Tracking, and Exercise Recommendations.

This plan ensures that BreatheWell meets its functional and non-functional requirements, adheres to defined standards, and provides a reliable, secure, and effective solution for lung-related health management.

2. Definitions and Acronyms

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **CI/CD:** Continuous Integration/Continuous Deployment
- **COPD:** Chronic Obstructive Pulmonary Disease
- **DB:** Database
- **IEEE:** Institute of Electrical and Electronics Engineers
- **ML:** Machine Learning
- **QA:** Quality Assurance
- **SQA:** Software Quality Assurance
- **SQAP:** Software Quality Assurance Plan
- **SRS:** System Requirements Specification
- **SDD:** Software Design Document
- **SME:** Subject Matter Expert
- **UI:** User Interface
- **UAT:** User Acceptance Testing
- **UX:** User Experience

3. Reference Documents

This SQAP is developed in accordance with and references the following documents:

- IEEE Standard 730:2014 – IEEE Standard for Software Quality Assurance Plans
- BreatheWell Project documentaion

4. SQA Plan Overview

This SQAP outlines a comprehensive approach to ensuring the quality of the BreatheWell application. Our strategy emphasizes proactive quality assurance throughout the development lifecycle, focusing on defect prevention, early detection, and continuous improvement. We are committed to delivering a reliable, secure, and user-friendly application that meets all specified requirements and user expectations.

4.1 Organization and Independence

The Software Quality Assurance (SQA) function for the BreatheWell project will be performed by a dedicated SQA team. This team operates independently from the software development team to ensure objectivity and impartiality in quality assessments.

- **Reporting Structure:** The SQA Lead will report directly to the Project Manager or a higher-level management authority (e.g., Head of Engineering, Director of Quality) to ensure that quality concerns are escalated and addressed without undue influence from development pressures.
- **Authority:** The SQA team will have the authority to:
 - Review and approve all project artifacts (requirements, design, code, test plans).
 - Conduct independent testing and audits.
 - Recommend process improvements.
 - Require retesting or re-inspection if quality criteria are not met.
 - Recommend halting a release if critical defects or non-conformances are identified that pose a significant risk to the system's functionality, security, or user safety.

4.2 Software Product Risk

The following risks have been identified as potentially impacting the quality of the BreatheWell software product, along with their proposed mitigation strategies:

Risk 1: Accuracy and Reliability of AI/ML Models

- **Description:** The core functionality relies heavily on the accuracy of machine learning models for cough/breathing analysis and smart diagnosis. Inaccurate predictions could lead to incorrect health insights or recommendations.
- **Mitigation:**
 - Rigorous validation of ML models using diverse and representative datasets.
 - Continuous monitoring of model performance in production.
 - Implementing A/B testing for model updates.
 - Involving medical SMEs in model evaluation and result interpretation.
 - Clear disclaimers to users that the application provides insights, not medical

diagnoses.

Risk 2: Data Security and Privacy of Sensitive Health Information

- **Description:** The application handles sensitive user health data (symptoms, audio recordings, medical history). Breaches could lead to severe privacy violations and legal repercussions.
- **Mitigation:**
 - Implement end-to-end encryption for data in transit and at rest.
 - Adhere to industry best practices for data security (e.g., OWASP Top 10).
 - Regular security audits, penetration testing, and vulnerability assessments.
 - Strict access controls and authentication mechanisms (Firebase).
 - Compliance with relevant data protection regulations (e.g., HIPAA considerations, GDPR considerations).

Risk 3: Integration Complexity (Frontend, Backend, Databases, Firebase, External APIs)

- **Description:** The full-stack architecture involves multiple technologies (React.js, Python/Django/Flask, PostgreSQL, MongoDB, Firebase, external air quality APIs), increasing the risk of integration issues.
- **Mitigation:**
 - Develop clear API contracts and documentation between frontend and backend.
 - Implement robust error handling and logging across all layers.
 - Conduct incremental integration testing throughout development.
 - Utilize CI/CD pipelines to automate integration and detect issues early.

Risk 4: Scalability and Performance under Load (1.3.2, 1.3.4)

- **Description:** High user traffic or large volumes of audio/symptom data submission could lead to performance bottlenecks, slow response times, or system crashes.
- **Mitigation:**
 - Design for horizontal scalability for both frontend and backend services.
 - Optimize database queries and indexing.
 - Conduct comprehensive performance, load, and stress testing.
 - Implement caching mechanisms where appropriate.

Risk 5: Usability and User Experience (UI/UX) (All features)

- **Description:** A complex or non-intuitive interface could deter users, especially given the health-related context.
- **Mitigation:**
 - Conduct early and continuous usability testing with target user groups.
 - Gather user feedback through surveys and interviews.
 - Iterative UI/UX design based on user feedback.
 - Adherence to established UI/UX design guidelines.

4.3 Tools

The following tools will be utilized to support SQA activities for the BreatheWell project:

- **Requirements Management:** Jira (for capturing, tracking, and linking requirements to test cases)

- **Test Management:** Jira (for test case creation, organization, execution tracking, and reporting)
- **Defect Tracking:** Jira (for logging, tracking, prioritizing, and managing defects)
- **Version Control:** Git / GitHub (for source code management and versioning of all project artifacts)
- **Static Code Analysis:**
 - Python: Pylint, Black, Flake8, SonarQube
 - JavaScript/React: ESLint, Prettier, SonarQube
- **Unit Testing Frameworks:**
 - Python: Pytest
 - React.js: Jest, React Testing Library
- **Integration Testing:** Postman (for API testing), Pytest with requests library
- **UI Automation Testing:** Selenium, Cypress (for end-to-end web application testing)
- **Performance Testing:** JMeter, Locust (for load, stress, and scalability testing)
- **Security Testing:** OWASP ZAP (for automated vulnerability scanning), manual penetration testing tools
- **CI/CD Pipeline:** Jenkins, GitHub Actions (for automated builds, tests, and deployments)
- **Monitoring & Logging:** Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana) for production monitoring and anomaly detection.

4.4 Standards, Practices, Conventions, and Metrics

- **Software Engineering Standards:**
 - Adherence to IEEE Standard 730:2014 for Software Quality Assurance Plans.
 - Compliance with internal company software development lifecycle (SDLC) processes.
- **Coding Practices and Conventions:**
 - **Python:** PEP 8 style guide, consistent naming conventions for variables, functions, classes.
 - **React.js:** Airbnb JavaScript Style Guide, consistent component naming and file structure.
 - **General:** Use of meaningful comments, clear function/method signatures, and avoidance of magic numbers.
 - **Code Reviews:** Mandatory peer code reviews for all significant code changes before merging to main branches.
- **Documentation Standards:**
 - All project documentation (SRS, SDD, Test Plans, User Manuals) will follow predefined templates and version control.
 - API documentation (e.g., OpenAPI/Swagger) will be maintained for backend services.
- **Quality Metrics:**
 - **Defect Density:** Number of defects per KLOC (Thousand Lines of Code) or per functional point.
 - **Defect Removal Efficiency (DRE):** $(\text{Number of defects found before release} / \text{Total defects found}) \times 100$.
 - **Test Coverage:** Percentage of code lines, branches, or functions covered by automated

tests.

- **Requirements Traceability:** Percentage of functional requirements linked to corresponding test cases.
- **Mean Time To Detect (MTTD):** Average time from defect injection to detection.
- **Mean Time To Resolve (MTTR):** Average time from defect detection to resolution.
- **Automated Test Pass Rate:** Percentage of automated tests passing in CI/CD pipeline.
- **Performance Metrics:** Response times for key transactions, system throughput, resource utilization.
- **AI Model Accuracy:** Specific metrics relevant to ML models (e.g., F1-score, precision, recall, AUC for classification tasks).

4.5 Effort, Resources, and Schedule

- **Personnel:**
 - **1 SQA Lead:** Responsible for overall SQA strategy, planning, team management, and reporting.
 - **2-3 SQA Engineers:** Responsible for test case design, manual and automated test execution, defect reporting, and various quality assurance activities.
 - **1 Performance & Security Test Specialist (part-time/on-demand):** For specialized non-functional testing.
- **Effort Allocation:** Approximately 15-20% of the total project development effort will be dedicated to SQA activities. This allocation will be adjusted based on project phase, complexity, and identified risks.
- **Resources:**
 - Dedicated test environments (development, staging, production-like).
 - Access to all necessary SQA tools and licenses.
 - Sufficient computing resources for automated test execution and performance testing.
 - Access to relevant documentation and development team members for clarification.
- **Schedule Integration:** SQA activities will be fully integrated into the project's Agile (Scrum) development sprints.
 - **Sprint Planning:** SQA team participates to estimate testing effort, identify testable stories, and define acceptance criteria.
 - **In-Sprint Testing:** Continuous testing (unit, integration, functional, regression) of features developed within the sprint.
 - **Sprint Review:** SQA team presents testing results and quality status of completed features.
 - **Release Planning:** Dedicated SQA cycles for system testing, UAT, performance, and security testing prior to major releases.
 - **Post-Deployment:** Monitoring, defect analysis, and feedback for continuous improvement.

5. Activities, Outcomes, and Tasks

This section details the specific quality assurance activities, their expected outcomes, and the tasks required to achieve them.

5.1 Product Assurance

Product assurance activities focus on ensuring that the BreatheWell software product conforms to its specified requirements, design, and user expectations.

- **Activity: Requirements Review**

- **Description:** Review of the SRS to ensure requirements are clear, unambiguous, complete, consistent, traceable, and testable.
- **Tasks:**
 - Participate in requirements elicitation and analysis meetings.
 - Conduct formal reviews of the SRS with stakeholders.
 - Identify and document any ambiguities, inconsistencies, or missing information.
 - Ensure requirements are verifiable and measurable.
- **Outcomes:** Reviewed SRS with documented comments and clarifications, updated requirements, and a clear understanding of testing scope.

- **Activity: Design Review**

- **Description:** Review of the Software Design Document (SDD), architectural diagrams, database schemas, and API specifications.
- **Tasks:**
 - Assess design for adherence to requirements, scalability, security, maintainability, and performance.
 - Verify proper integration points between frontend, backend, databases, and external services.
 - Identify potential design flaws or architectural risks.
- **Outcomes:** Reviewed SDD with feedback, identified design issues, and proposed design improvements.

- **Activity: Code Review & Static Analysis**

- **Description:** Examination of source code to identify defects, coding standard violations, security vulnerabilities, and logic errors.
- **Tasks:**
 - Conduct peer code reviews for all major feature implementations.
 - Utilize static code analysis tools (SonarQube, ESLint, Pylint) to enforce coding standards and detect common issues.
 - Ensure proper commenting and documentation within the code.
- **Outcomes:** Code review reports, static analysis findings, improved code quality, and reduced defect injection rate.

- **Activity: Testing (Comprehensive)**

- **Description:** Execution of various testing types to verify functionality, performance, security, and usability.
- **Tasks:**
 - **Test Planning:** Develop detailed test plans for each sprint and release, outlining scope, objectives, entry/exit criteria, and resources.
 - **Test Case Design:** Create detailed, executable test cases based on functional requirements (1.3.1-1.3.9), non-functional requirements, and design

specifications.

- **Test Environment Setup:** Configure and maintain stable and representative test environments.
- **Unit Testing:** Developers write and execute unit tests for individual components/modules. SQA reviews unit test coverage.
- **Integration Testing:** Verify the interactions between different modules, components, and services (e.g., frontend-backend API calls, backend-database interactions, Firebase integration).
- **Functional Testing:** Systematically test each feature (Symptom Submission, AI Analysis, Smart Diagnosis, Dashboard, Care Plan, Environmental Alert, Chatbot, Doctor Appointment, Exercise Recommendations) against its specified requirements.
- **Regression Testing:** Execute automated and manual regression test suites to ensure new changes do not negatively impact existing functionality.
- **AI Model Validation Testing:**
 - Specific testing for 1.3.2 (AI-Powered Cough & Breathing Analysis) and 1.3.3 (Smart Diagnosis System).
 - Use diverse datasets (including edge cases and adversarial examples) to evaluate model accuracy, precision, recall, and robustness.
 - Test model performance under various audio qualities and background noises.
 - Verify the integration of ML model outputs into the smart diagnosis system.
- **Performance Testing:** Conduct load, stress, and scalability tests on critical paths (e.g., audio upload/analysis, dashboard loading, real-time alerts) to identify bottlenecks and ensure system responsiveness under expected and peak loads.
- **Security Testing:** Perform vulnerability scanning, penetration testing, authentication/authorization testing, and data privacy compliance checks.
- **Usability Testing:** Conduct user studies and heuristic evaluations to assess the ease of use, intuitiveness, and overall user satisfaction of the web interface.
- **Compatibility Testing:** Test the application across various web browsers (Chrome, Firefox, Edge, Safari) and devices (desktop, tablet, mobile) to ensure consistent user experience.
- **Defect Management:** Log, prioritize, track, and retest all identified defects using Jira.
- **Outcomes:** Comprehensive test plans, detailed test cases, test execution reports, defect logs, performance test reports, security test reports, usability feedback, and a final test summary report indicating readiness for deployment.
- **Activity: Acceptance Testing (UAT)**
 - **Description:** Formal testing by end-users or product owners to validate that the system meets business requirements and is fit for purpose.
 - **Tasks:**
 - Prepare UAT test scenarios and scripts.
 - Facilitate UAT sessions with designated users.

- Collect and document user feedback and identified issues.
- Obtain formal sign-off from stakeholders.
- **Outcomes:** UAT test reports, identified user experience improvements, and formal acceptance of the system for release.

5.2 Process Assurance

Process assurance activities verify that the software development processes themselves are followed according to defined standards and procedures, contributing to overall product quality.

- **Activity: Process Monitoring**

- **Description:** Continuous monitoring of adherence to the defined software development lifecycle processes (e.g., Agile Scrum, CI/CD workflows, change management procedures).
- **Tasks:**
 - Regularly review project management tools (Jira) to ensure tasks are progressing as per defined workflows.
 - Verify that development teams are following coding standards, conducting peer reviews, and committing code regularly.
 - Ensure proper use of version control and branching strategies.
- **Outcomes:** Process compliance reports, identification of process deviations, and recommendations for process adherence and improvement.

- **Activity: Audits**

- **Description:** Periodic, systematic examinations of project artifacts and processes to determine compliance with established standards, policies, and plans.
- **Tasks:**
 - Conduct internal audits of requirements, design documents, test plans, code, and defect logs.
 - Verify traceability between requirements, design, code, and test cases.
 - Assess the effectiveness of configuration management and change control.
- **Outcomes:** Audit reports with findings, non-conformances, corrective action plans, and verification of action item closure.

- **Activity: Configuration Management Review**

- **Description:** Verification that all software configuration items (source code, documentation, test scripts, build scripts, deployment configurations) are under version control and properly managed.
- **Tasks:**
 - Ensure baselines are established for each release.
 - Verify that changes are properly controlled, tracked, and approved.
 - Confirm the integrity and consistency of the configuration management system.
- **Outcomes:** Configuration management audit reports, ensuring the integrity and traceability of all project assets throughout the lifecycle.

6. Additional Considerations

6.1 Contract Review

For any external services, third-party components, or vendor engagements (e.g., specialized ML model development, external security audits, cloud hosting providers), the SQA Lead, in conjunction with legal and procurement teams, will review contracts to ensure that quality requirements, service level agreements (SLAs), data security clauses, and intellectual property rights are clearly defined, measurable, and enforceable. Specific quality gates and acceptance criteria for vendor deliverables will be outlined.

6.2 Quality Measurement

Quality metrics, as defined in Section 4.4, will be collected and analyzed continuously. Data will be gathered from various sources, including test management tools, defect tracking systems, static analysis reports, and production monitoring systems.

- **Collection:** Automated collection via CI/CD pipelines where possible (e.g., test coverage, build success rate). Manual collection for reviews, audits, and specific test types.
- **Evaluation:** Regular analysis (e.g., weekly SQA meetings, sprint reviews, monthly project reviews) to identify trends, pinpoint areas of concern, and assess overall product and process quality.
- **Reporting:** Quality dashboards will be maintained to visualize key metrics. Reports will be generated for project stakeholders to inform decision-making regarding releases, resource allocation, and process improvements.

6.3 Waivers and Deviations

Any proposed deviation from this SQAP, established standards, or defined processes must be formally documented.

- **Process:** A formal waiver request must be submitted to the SQA Lead and Project Manager. The request must include:
 - Description of the deviation.
 - Justification for the deviation.
 - Assessment of potential impact on quality, schedule, and cost.
 - Proposed mitigation strategies for any identified risks.
- **Approval:** Waivers require written approval from the SQA Lead and Project Manager. For significant deviations affecting critical quality attributes or regulatory compliance, approval from higher management or relevant stakeholders (e.g., Product Owner) may be required.
- **Records:** All approved waivers and deviations will be recorded in the SQA records.

6.4 Task Repetition

Tasks will be repeated under the following conditions:

- **Retesting:** After a defect is fixed, targeted retesting will be performed to confirm the fix and ensure no new defects have been introduced in the affected area.
- **Regression Testing:** A full or partial regression test suite will be executed after major code changes, new feature implementations, significant bug fixes, environment changes, or

prior to any major release to ensure existing functionality remains intact.

- **Re-inspections/Re-reviews:** If a document (e.g., SRS, SDD) or code module undergoes significant changes after its initial review, or if the initial review identified a high number of critical issues, a re-inspection or re-review will be conducted.
- **Re-audits:** If significant non-conformances are identified during a process or product audit, follow-up audits will be scheduled to verify the effectiveness of corrective actions.

6.5 Risk to Performing SQA

Risks that may prevent or impact the effective execution of SQA activities include:

- **Insufficient SQA Resources:** Lack of adequate SQA personnel, tools, or budget.
 - **Handling:** Proactive resource planning, clear justification of SQA value to management, and prioritization of SQA activities based on risk.
- **Compressed Schedules:** Aggressive project timelines that limit time for thorough SQA activities.
 - **Handling:** Early identification of schedule risks, clear communication of quality vs. schedule trade-offs, and advocating for realistic timelines.
- **Scope Creep:** Uncontrolled changes to requirements or features that lead to an expanding scope without corresponding adjustments in SQA effort.
 - **Handling:** Strict change control procedures, clear definition of project scope, and regular communication with stakeholders regarding scope changes and their impact.
- **Lack of Access to SMEs/Test Data:** Difficulty in obtaining timely access to medical SMEs for AI model validation or insufficient representative test data.
 - **Handling:** Establish clear communication channels with SMEs, plan for data acquisition and anonymization early, and collaborate with data scientists to generate synthetic data if necessary.
- **Technical Complexity of AI/ML:** Challenges in thoroughly testing and validating complex, evolving AI models.
 - **Handling:** Invest in specialized training for SQA engineers on AI/ML testing, collaborate closely with ML engineers, and leverage advanced testing techniques for AI.

6.6 Communication Strategy

Effective communication is crucial for successful SQA. SQA-related information will be communicated through:

- **Regular Meetings:**
 - Daily SQA team stand-ups.
 - Participation in daily development team stand-ups.
 - Sprint review meetings (reporting on quality status of completed features).
 - Weekly SQA status meetings with the Project Manager and key stakeholders.
 - Ad-hoc meetings for critical issue discussions.
- **Reports:**
 - Weekly SQA Status Reports (progress, key findings, risks).
 - Sprint Quality Reports (after each sprint).

- Defect Trend Reports.
- Test Summary Reports (prior to major releases).
- Audit Reports.
- **Tools:**
 - Jira (or similar) for real-time defect status, test case execution status, and traceability.
 - Shared documentation platforms (e.g., Confluence) for SQA plans, procedures, and guidelines.
- **Escalation Procedures:** A defined escalation matrix will be followed for critical quality issues that cannot be resolved at the team level, ensuring timely attention from appropriate management.

6.7 Non-Conformance Process

A formal process will be followed for identifying, recording, reporting, tracking, and resolving all non-conformances (e.g., software defects, process deviations, documentation errors).

1. **Identification:** A non-conformance is identified during any SQA activity (testing, review, audit) or reported by users.
2. **Recording:** The non-conformance is immediately logged in the defect tracking system (Jira), including a unique ID, detailed description, steps to reproduce (for defects), actual vs. expected results, severity, priority, and relevant attachments (screenshots, logs).
3. **Reporting:** The logged non-conformance is automatically visible to relevant teams. Critical non-conformances are communicated immediately to the development team lead and Project Manager.
4. **Analysis:** The development team investigates the non-conformance to determine its root cause and identify a resolution.
5. **Resolution:** The development team implements the fix or corrective action.
6. **Tracking:** The status of the non-conformance is continuously tracked in Jira (e.g., Open, In Progress, Resolved, Ready for Retest, Closed).
7. **Verification:** The SQA team retests the fixed non-conformance to confirm its resolution and performs regression testing to ensure no new issues were introduced.
8. **Closure:** Once verified and accepted by SQA, the non-conformance is formally closed in the tracking system.

7. SQA Records

This section specifies how records of SQA activities are maintained, secured, and made available for review.

7.1 Analyze, Identify, Collect, File, Maintain

All SQA-related records will be systematically analyzed, identified, collected, filed, and maintained to provide objective evidence of quality assurance activities, support traceability, and facilitate continuous process improvement. These records include, but are not limited to:

- **Planning Documents:** This SQAP, Test Plans, Test Strategies.
- **Requirements & Design Artifacts:** Reviewed SRS, SDD, DBDD, UI/UX mockups, API

specifications.

- **Test Artifacts:** Detailed Test Cases (manual and automated scripts), Test Data, Test Execution Logs, Test Summary Reports.
- **Defect Management Records:** All logged defect reports (including status, resolution, and retest results), defect trend analysis.
- **Review & Audit Reports:** Code review checklists and findings, static analysis reports, SQA audit reports, process compliance reports.
- **Configuration Management Records:** Baselines, change requests, version control logs.
- **Communication Records:** Meeting minutes, decision logs, formal communications related to quality.
- **Waivers & Deviations:** Approved waiver requests and their justifications.
- **Performance & Security Test Results:** Detailed reports from specialized testing.
- **UAT Reports:** User feedback and formal sign-offs.

7.2 Availability of Records

All SQA records will be stored electronically in a centralized, secure, and version-controlled repository (e.g., project management system like Jira, shared network drives, cloud storage with access controls).

- **Storage:** Records will be organized logically by project phase and artifact type.
- **Access Control:** Access to SQA records will be restricted based on roles and responsibilities, ensuring confidentiality and integrity.
- **Retention:** Records will be retained according to organizational retention policies and any applicable regulatory requirements concerning health data (e.g., HIPAA considerations, GDPR considerations) for a minimum period of 7 years or as legally mandated.
- **Availability:** Records will be readily accessible to authorized personnel for internal audits, external compliance checks, project reviews, and post-mortem analysis.